```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

data = pd.read_csv("Credit Score Classification Dataset.csv")

print("CREDIT SCORE CLASSIFICATION")
print("---------------------------")

print("\nFirst 5 Records:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nColumn Names:")
print(data.columns)

print("\nMissing Values:")
print(data.isnull().sum())

data = data.dropna()

encoder = LabelEncoder()

categorical_columns = [
    "Gender",
    "Education",
    "Marital Status",
    "Home Ownership"
]

for column in categorical_columns:
    data[column] = encoder.fit_transform(data[column])

data["Credit Score"] = encoder.fit_transform(data["Credit Score"])

X = data.drop("Credit Score", axis=1)
y = data["Credit Score"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nActual Values:")
print(y_test.values)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy Percentage:")
print(accuracy * 100, "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

new_customer = pd.DataFrame({
    "Age": [30],
    "Gender": [1],
    "Income": [75000],
    "Education": [1],
    "Marital Status": [1],
    "Number of Children": [1],
    "Home Ownership": [0]
})
```

```python
prediction = model.predict(new_customer)

print("\nNew Customer Prediction:")
print(prediction[0])

if prediction[0] == 0:
    print("Credit Score: Low")
elif prediction[0] == 1:
    print("Credit Score: Average")
else:
    print("Credit Score: High")
```

```
4                   3          Owned         High

Dataset Shape:
(164, 8)

Column Names:
Index(['Age', 'Gender', 'Income', 'Education', 'Marital Status',
       'Number of Children', 'Home Ownership', 'Credit Score'],
      dtype='object')

Missing Values:
Age                   0
Gender                0
```

```
CREDIT SCORE CLASSIFICATION
---------------------------

First 5 Records:
   Age  Gender  Income            Education Marital Status  \
0   25  Female   50000    Bachelor's Degree         Single
1   30    Male  100000      Master's Degree        Married
2   35  Female   75000            Doctorate        Married
3   40    Male  125000  High School Diploma         Single
4   45  Female  100000    Bachelor's Degree        Married

   Number of Children Home Ownership Credit Score
0                   0         Rented         High
1                   2          Owned         High
2                   1          Owned         High
3                   0          Owned         High
4                   3          Owned         High
```